

compact_tree_models

May 27, 2026

1 Compact Feature Modeling (Non-Leaky)

This notebook builds a compact feature table (bank features + row-level summaries + top MI / missingness-gap columns) and trains XGBoost + LightGBM.

```
[8]: from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.feature_selection import mutual_info_classif
from sklearn.impute import SimpleImputer
from sklearn.metrics import average_precision_score, confusion_matrix, \
    roc_auc_score
from sklearn.model_selection import train_test_split

RANDOM_STATE = 42
DATA_PATH = Path('DataSet.csv')
TARGET_COL = 'F3924'
ID_COL = 'Unnamed: 0'
LEAKY_FEATURES = ['F3912']

BANK_FEATURES = [
    'F115', 'F321', 'F527', 'F531', 'F670', 'F1692', 'F2082', 'F2122',
    'F2582', 'F2678', 'F2737', 'F2956', 'F3043', 'F3836', 'F3887',
    'F3889', 'F3891', 'F3894'
]

PLACEHOLDER_VALUES = {-99999999, 99999999, -9999999, 9999999, -999999, 999999, \
    -9999, 9999}
LARGE_ABS_THRESHOLD = 1e7
PLACEHOLDER_MIN_FRAC = 0.002

TOP_MI = 25
TOP_GAP = 25

[9]: df = pd.read_csv(DATA_PATH)
print(f'Loaded shape: {df.shape}')
```

```

if ID_COL in df.columns:
    df = df.drop(columns=[ID_COL])

y = df[TARGET_COL].astype(int)
X = df.drop(columns=[TARGET_COL], errors='ignore')

if LEAKY_FEATURES:
    X = X.drop(columns=[col for col in LEAKY_FEATURES if col in X.columns],
        errors='ignore')

X = X.replace(list(PLACEHOLDER_VALUES), np.nan)
X = X.replace([np.inf, -np.inf], np.nan)
X = X.apply(pd.to_numeric, errors='coerce')

def detect_placeholder_values(frame: pd.DataFrame, abs_threshold: float,
    min_frac: float) -> dict[str, float]:
    placeholder_map: dict[str, float] = {}
    for col in frame.columns:
        series = frame[col].dropna()
        if series.empty:
            continue
        extreme = series[series.abs() >= abs_threshold]
        if extreme.empty:
            continue
        counts = extreme.value_counts()
        candidate = counts.index[0]
        if counts.iloc[0] / len(series) >= min_frac:
            placeholder_map[col] = candidate
    return placeholder_map

placeholder_map = detect_placeholder_values(X, LARGE_ABS_THRESHOLD,
    PLACEHOLDER_MIN_FRAC)
for col, value in placeholder_map.items():
    X[col] = X[col].replace(value, np.nan)

print('Features after cleaning:', X.shape)
print('Placeholder columns detected:', len(placeholder_map))
print('Target base rate:', y.mean())

```

```

Loaded shape: (9082, 3925)
Features after cleaning: (9082, 3922)
Placeholder columns detected: 76
Target base rate: 0.008918740365558247

```

```

[10]: def build_row_stats(frame: pd.DataFrame) -> pd.DataFrame:
        values = frame.to_numpy(dtype=float)
        mask = ~np.isnan(values)

```

```

non_missing = mask.sum(axis=1)
total = values.shape[1]
missing_rate = 1.0 - (non_missing / total)

zero_rate = np.where(non_missing > 0, (values == 0).sum(axis=1) /
↳non_missing, 0)
positive_rate = np.where(non_missing > 0, (values > 0).sum(axis=1) /
↳non_missing, 0)
negative_rate = np.where(non_missing > 0, (values < 0).sum(axis=1) /
↳non_missing, 0)

with np.errstate(all='ignore'):
    mean = np.nanmean(values, axis=1)
    std = np.nanstd(values, axis=1)
    min_val = np.nanmin(values, axis=1)
    max_val = np.nanmax(values, axis=1)
    median = np.nanmedian(values, axis=1)
    q25 = np.nanpercentile(values, 25, axis=1)
    q75 = np.nanpercentile(values, 75, axis=1)
    abs_mean = np.nanmean(np.abs(values), axis=1)

iqr = q75 - q25

return pd.DataFrame({
    'row_non_missing_count': non_missing,
    'row_missing_rate': missing_rate,
    'row_zero_rate': zero_rate,
    'row_positive_rate': positive_rate,
    'row_negative_rate': negative_rate,
    'row_mean': mean,
    'row_std': std,
    'row_min': min_val,
    'row_max': max_val,
    'row_median': median,
    'row_q25': q25,
    'row_q75': q75,
    'row_iqr': iqr,
    'row_abs_mean': abs_mean,
}, index=frame.index)

row_stats = build_row_stats(X)
row_stats.head()

```

```

[10]:   row_non_missing_count  row_missing_rate  row_zero_rate  row_positive_rate  \
0                2728          0.304437        0.488636        0.294355
1                2776          0.292198        0.482709        0.293948
2                2780          0.291178        0.451439        0.330576

```

3	2900	0.260581	0.423793	0.378966
4	2788	0.289138	0.463773	0.330703

	row_negative_rate	row_mean	row_std	row_min	row_max \
0	0.217009	17724.644139	243015.108810	-1.00	5933313.82
1	0.223343	15757.835375	230671.462377	-1.00	9133588.45
2	0.217986	6140.730902	46102.216009	-1.02	1093319.56
3	0.197241	29135.495296	246015.394471	-1.00	4458592.60
4	0.205524	7528.900142	62142.156960	-1.19	1154858.43

	row_median	row_q25	row_q75	row_iqr	row_abs_mean
0	0.0	0.0	0.530000	0.530000	17725.036867
1	0.0	0.0	0.400000	0.400000	15758.246823
2	0.0	0.0	0.978333	0.978333	6141.129866
3	0.0	0.0	1.000000	1.000000	29135.851875
4	0.0	0.0	0.768529	0.768529	7529.261570

```
[11]: bank_features = [col for col in BANK_FEATURES if col in X.columns and X[col].
    ↪notna().any()]

mi_candidates = X.loc[:, X.nunique(dropna=True) > 1]
top_mi_cols = []
if mi_candidates.shape[1] > 0:
    imputer = SimpleImputer(strategy='median')
    mi_values = imputer.fit_transform(mi_candidates)
    mi_scores = mutual_info_classif(mi_values, y, random_state=RANDOM_STATE)
    mi_series = pd.Series(mi_scores, index=mi_candidates.columns).
    ↪sort_values(ascending=False)
    top_mi_cols = mi_series.head(TOP_MI).index.tolist()

missing_pos = X.loc[y == 1].isna().mean()
missing_neg = X.loc[y == 0].isna().mean()
missing_gap = (missing_pos - missing_neg).abs()
missing_gap = missing_gap[missing_gap > 0].sort_values(ascending=False)
top_gap_cols = missing_gap.head(TOP_GAP).index.tolist()

selected_cols = []
for col in bank_features + top_mi_cols + top_gap_cols:
    if col not in selected_cols:
        selected_cols.append(col)

missing_flags = X[top_gap_cols].isna().astype(int)
missing_flags = missing_flags.add_prefix('miss_')

compact_df = pd.concat([X[selected_cols], row_stats, missing_flags], axis=1)
compact_df = compact_df.loc[:, compact_df.isna().mean() < 1.0]
```

```

print('Bank features kept:', len(bank_features))
print('Top MI columns:', len(top_mi_cols))
print('Top missingness-gap columns:', len(top_gap_cols))
print('Compact feature shape:', compact_df.shape)

```

```

Bank features kept: 16
Top MI columns: 25
Top missingness-gap columns: 25
Compact feature shape: (9082, 104)

```

```

[12]: X_train, X_valid, y_train, y_valid = train_test_split(
    compact_df,
    y,
    test_size=0.2,
    stratify=y,
    random_state=RANDOM_STATE
)

scale_pos_weight = (y_train == 0).sum() / max((y_train == 1).sum(), 1)
print('Train shape:', X_train.shape)
print('Valid shape:', X_valid.shape)
print('Scale pos weight:', scale_pos_weight)

def report_metrics(name: str, y_true: pd.Series, y_prob: np.ndarray) -> None:
    pr_auc = average_precision_score(y_true, y_prob)
    roc_auc = roc_auc_score(y_true, y_prob)
    print(f'{name} PR-AUC: {pr_auc:.4f} | ROC-AUC: {roc_auc:.4f} | base rate: {y_true.mean():.4f}')
    y_pred = (y_prob >= 0.5).astype(int)
    cm = confusion_matrix(y_true, y_pred)
    print('Confusion matrix (threshold=0.5):')
    print(cm)

```

```

Train shape: (7265, 104)
Valid shape: (1817, 104)
Scale pos weight: 110.76923076923077

```

```

[13]: try:
    import xgboost as xgb
    has_xgb = True
except ImportError:
    has_xgb = False
    print('xgboost not installed. Install with: pip install xgboost')

if has_xgb:
    xgb_model = xgb.XGBClassifier(
        n_estimators=500,
        max_depth=5,

```

```

        learning_rate=0.05,
        subsample=0.8,
        colsample_bytree=0.8,
        eval_metric='aucpr',
        scale_pos_weight=scale_pos_weight,
        tree_method='hist',
        random_state=RANDOM_STATE,
        n_jobs=-1,
    )
    xgb_model.fit(X_train, y_train)
    xgb_prob = xgb_model.predict_proba(X_valid)[:, 1]
    report_metrics('XGBoost', y_valid, xgb_prob)

```

XGBoost PR-AUC: 0.8884 | ROC-AUC: 0.9984 | base rate: 0.0088

Confusion matrix (threshold=0.5):

```

[[1797   4]
 [   5  11]]

```

```

[14]: try:
        import lightgbm as lgb
        has_lgb = True
    except ImportError:
        has_lgb = False
        print('lightgbm not installed. Install with: pip install lightgbm')

    if has_lgb:
        lgb_model = lgb.LGBMClassifier(
            n_estimators=500,
            learning_rate=0.05,
            num_leaves=31,
            subsample=0.8,
            colsample_bytree=0.8,
            objective='binary',
            scale_pos_weight=scale_pos_weight,
            random_state=RANDOM_STATE,
            n_jobs=-1,
        )
        lgb_model.fit(X_train, y_train)
        lgb_prob = lgb_model.predict_proba(X_valid)[:, 1]
        report_metrics('LightGBM', y_valid, lgb_prob)

```

[LightGBM] [Info] Number of positive: 65, number of negative: 7200

[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.004351 seconds.

You can set `force\_col\_wise=true` to remove the overhead.

[LightGBM] [Info] Total Bins 15634

[LightGBM] [Info] Number of data points in the train set: 7265, number of used features: 103

[illegible]

[illegible]

[illegible]

[illegible]

